Data Structures(202040301)

Unit:-1

Introduction to Data Structure

Outline

- Data Management Concepts
- Data Types – Primitive and Non-primitive
- Types of Data Structures
- Performance Analysis and Measurement

Data Management Concepts

Why to Learn Data Structures ?

- **Computer** is an electronic machine which is used for **data processing and manipulation.**
- When programmer collects such type of data for processing, he would require to store all of them in computer's main memory.
- **In order to make computer work we need to know**
  - Representation of data in computer
  - Accessing of data
  - How to solve problem step by step
    - For doing this task we use data structure

Continue…

- There are **three common problems that applications face** now-a-days as applications are getting complex and data rich

  - **Data Search** – Consider **an inventory of 1 million(10^6) items of a store**. If the application is to search an item, it has to search an item in 1 million(10^6) items every time slowing down the search. As data grows, search will become slower.
  - **Processor speed** – Processor speed although being very high, falls limited if the data grows to billion records.
  - **Multiple requests** – As thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.

- To solve the above-mentioned problems, data structure comes into the picture. **Data can be organized in a data structure in such a way that all items may not be required to be searched**, and the required data can be searched instantly.

Introduction to Data Structure

- **Data:** Data is the collection of numbers, alphabets and symbols to represent information. Data means value or set of values**.(eg.marks of students, name of employee, address of a customer, value of *Pi, etc.*)**
  - It is raw form of information. Processed data is called information.
- **Record:** A record is a collection of data items. **For example, the name, address, course and marks obtained are individual data items.** But all these items can be grouped together to form a record.
- **File:** A file is a collection of related records. For example, if there are 60 students in a class, then there are 60 records of students. All these related records are stored in a file.
- **Algorithm:** Algorithm is the finite set of instructions to solve a particular problem.
  - **Algorithm + Data Structure = Program**

Continue…

- **Data Structure:** Data structure is basically a group of data elements that are put together under one name to define a particular way of storing and organizing data in a computer so that it can be used efficiently.

  - Data Structure is a systematic way to organize data in computer memory in order to use it efficiently.

  - The way in which the data is organized affects the performance of a program for different tasks.

  - Some of the more commonly used data structures are array, linked list, queue, stack, tree, graph and hash table.

Continue…

- **Data structure study covers the following points**
  - Amount of memory require to store
  - Amount of time require to process
  - Representation of data in memory
  - Operations performed on that data

- **Characteristics of a Data Structure**
- **Correctness** – Data structure implementation should implement its operations correctly.
- **Time Complexity** – Running time or the execution time of operations of data structure must be as small as possible.
- **Space Complexity** – Memory usage of a data structure operation should be as little as possible.

Applied areas for Data Structures

- Compiler Design
- Statistical Analysis Package
- Numerical Analysis
- Artificial Intelligence
- Operating System
- DBMS
- Simulation
- Graphics

Types of Data Types

Primitive and Non-primitive

Data Types – Primitive and Non-Primitive

- A **data type** is a classification of data, which can store a specific type of information.
  - When a variable is declared, it is assigned a data type that determines what type of data the variable may contain.

1) Primitive Data type:

- The term 'Data type', 'basic data type', 'primitive data type' 'Built-in data type' are often used interchangeably.
- Primitive data types are predefined types of data, which are supported by C language.
- ***Examples of Such data types include int, char, float & double.***

Data Types – Primitive and Non-Primitive

(Continue…)

- **Integer:** Integer represents numerical values which are whole quantities.
  - The number of objects are countable can be represented by an integer.
  - The set of integer is:{…..-(n+1), -n , ….. , -2, -1,0,1,2,……n,n+1}
  - The sign and magnitude method is used to represent integer numbers. In this method place a sign symbol in front of the number.
  - **Ex: 100,56,-36,-345,0,+467**
- **Real(float):** The number having fractional part i.e decimal point is called real number.
  - In this method the real number is expressed as a combination of mantissa and exponent.
  - **Ex: 0.004,-0.34,645.9,0.56,0.4e-2,-3.4e-1123.45,12.9e14**
- **Character:** Character data type is used to store nonnumeric information.
  - It can be letters [A-Z], [a-z], operators and special symbols.
  - A character is represented in memory as a sequence bits.
  - Two popular character set supported by computer are ASCII and EBCDIC.

Data Types – Primitive and Non-Primitive

(Continue…)

2) Non-Primitive Data type:

- Non-primitive data types are those types of data that are not defined by C programming language, but are created by the programmer or user defined.
- The Non-Primitive data types are created using the basic data types.
- ***Examples of such data types include array, linked lists, stack, queues, graphs, etc.***
- A Non-Primitive data types are further divided into Linear & Non Linear Data Structure.
- **Array:** An array is a fixed-size sequenced collection of elements of the same data type.
  - Array is an ordered set which consist of a fixed number of object.

Data Types – Primitive and Non-Primitive

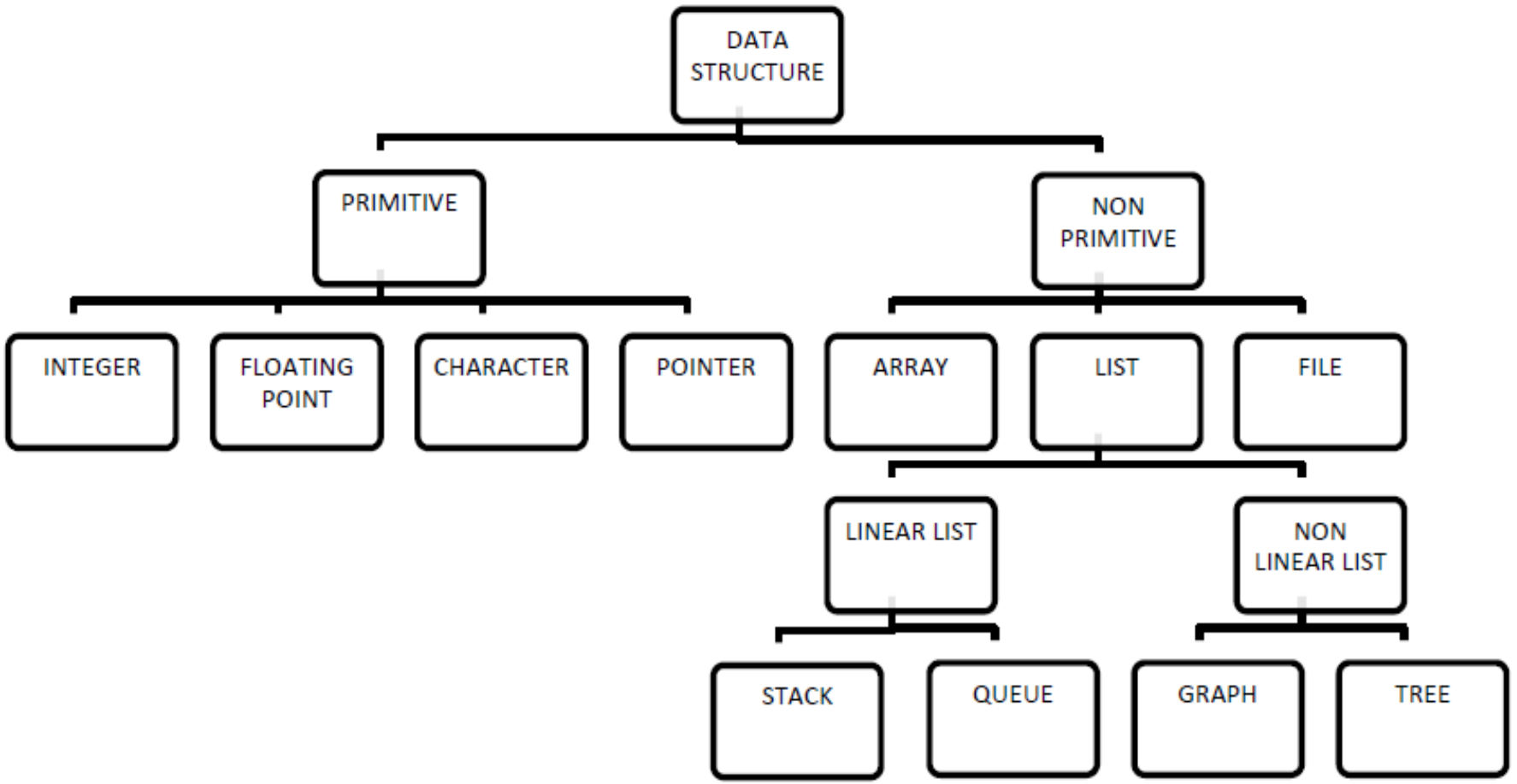
(Continue…)

- **List:** List is an ordered set which consist of variable number of elements or object.
  - An ordered set containing variable number of elements is called as Lists.
- **File:** A file is a collection of logically related information. It can be viewed as large list of record consisting of various field.
  - A file is a large list that is stored in the external memory of computer.
  - A file may be used as a repository for list items commonly called records.

Types of Data Structure

Linear and Non-Linear

Classification of Data Structures



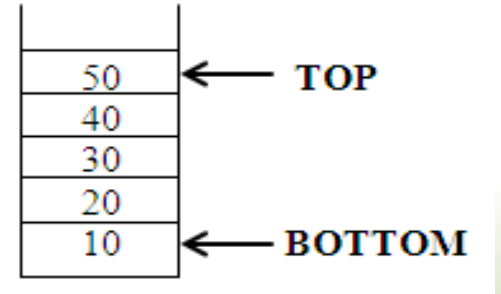
Types of Data Structures
Linear & Non Linear Data Structures

1)LINEAR DATA STRUCTURE:

- ***A data structure is said to be linear, if its elements are connected in linear fashion by mean of logically or in sequence memory locations.***
  - In the linear data structure processing of data items is possible in linear fashion.
  - Data are processed one by one sequentially.
  - Examples of the linear data structure are:
    - Array
    - Stack
    - Queue
    - Linked List

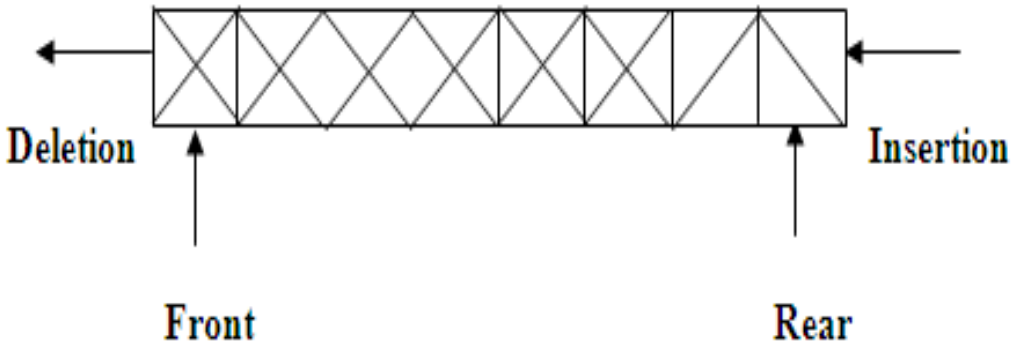
Types of Data Structures
Linear & Non Linear Data Structures
(Continue…)

- **Array:** Array is an ordered set which consist of a fixed number of object.
  - An array is a fixed-size sequenced collection of elements of the same data type
- **Stack:** Stack is a data structure in which insertion and deletion operations are performed at one end only.
  - The insertion operation is referred to as 'PUSH' and deletion operation is referred to as 'POP' operation.
  - Stack is also called as Last in First out (LIFO) data structure.



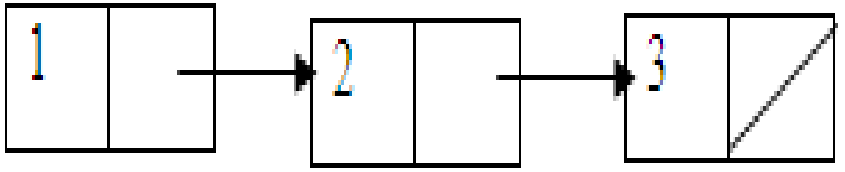
Types of Data Structures
Linear & Non Linear Data Structures
(Continue…)

- **Queue:** The data structure which permits the insertion at one end and Deletion at another end, known as Queue.
  - End at which deletion is occurs is known as FRONT end and another end at which insertion occurs is known as REAR end.
  - Queue is also called as First in First out (FIFO) data structure.



Types of Data Structures
Linear & Non Linear Data Structures
(Continue…)

- **Linked List:** A linked list is a collection of nodes.
  - Each node has two fields:
    - Information
    - Address, which contains the address of the next node.



Types of Data Structures
Linear & Non Linear Data Structures

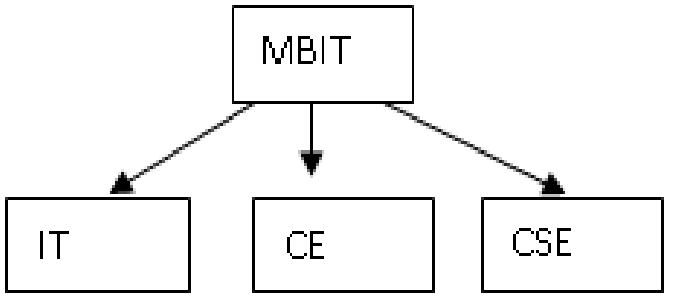
2) NON LINEAR DATA STRUCTURE:

- ***Nonlinear data structures are those data structure in which data items are not arranged in a sequence.***

  - Examples of Non-linear Data Structure are:
    - Tree
    - Graph

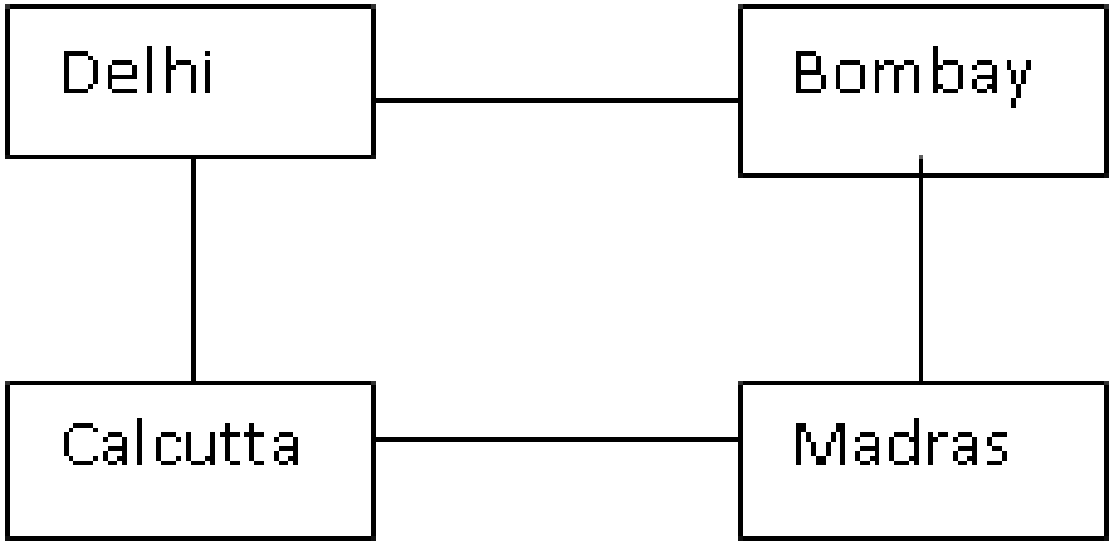
Types of Data Structures
Linear & Non Linear Data Structures

- **Tree:** A tree can be defined as finite set of data items (nodes) in which data items are arranged in branches and sub branches according to requirement.
  - Trees represent the hierarchical relationship between various elements.
  - Tree consists of nodes connected by edge, the node represented by circle and edge lives connecting to circle.



Types of Data Structures
Linear & Non Linear Data Structures

- **Graph:** Graph is a collection of nodes (Information) and connecting edges (Logical relation) between nodes.
  - A tree can be viewed as restricted graph.



Difference between Linear and Non- Linear Data Structure

Linear Data Structure	Non-Linear Data Structure
➢ Every item is related to its previous and next time.	➢ Every item is attached with many other items.
➢ Data is arranged in linear sequence.	➢ Data is not arranged in sequence.
➢ Data items can be traversed in a single run.	➢ Data cannot be traversed in a single run.
➢ Eg. Array, Stacks, linked list, queue.	➢ Eg. tree, graph.
➢ Implementation is easy.	➢ Implementation is difficult.

Operations on Data Structures

- **Insertion:** It is used to add new data items to the given list of data items.
  - For example to add the details of a new student who has recently joined the course. The create operation results in reserving memory for program elements.

- **Deletion:** It is used to remove a particular data item from the given collection of data items.
  - For example to add the details of a new student who has recently joined the course. The create operation results in reserving memory for program elements.

- **Traversal:** it means to access each data item exactly once so that it can be processed.
  - eg. To print the names of all the students in a class.

Continue…

- **Updation:** It updates or modifies the data in the data structure.
- **Searching:** It is used to find the location of one or more data items that satisfy the given condition. Such items may or may not be present in the given item list.
- **Sorting:** Sorting is a process of arranging all data items in a data structure in a particular order, either in ascending order or in descending order.
- **Merging:** Merging is a process of combining the data items of two different sorted list into a single sorted list.

Performance Analysis and Measurement

Performance Analysis and Measurement

Algorithm:

- Algorithm is a stepwise solution to a problem.
- Algorithm is a finite set of instructions that is followed to accomplish a particular task in a finite amount of time.
- In mathematics and computing, an algorithm is a procedure for accomplishing some task which will terminate in a defined end-state.
- The computational complexity and efficient implementation of the algorithm are important in computing, and this depends on suitable data structure
- Data structures are implemented using algorithms.
- An algorithm is a procedure that you can write as a C function or program, or any other language.
- An algorithm states explicitly how the data will be manipulated.

Performance Analysis and Measurement
(Continue…)

Algorithm Efficiency:

- Some algorithms are more efficient than others. We would prefer to choose an efficient algorithm, so it would be nice to have metrics for comparing algorithm efficiency.
- The complexity of an algorithm is a function describing the efficiency of the algorithm in terms of the amount of data the algorithm must process.
- Usually there are natural units for the domain and range of this function. There are two main complexity measures of the efficiency of an algorithm

Performance Analysis and Measurement
(Continue…)

Properties Required for Algorithm:

- **Finiteness:** An algorithm must always terminate after a finite number of steps. Every algorithm should have a proper end. The algorithm cant lead to an infinite condition.
- **Definiteness:** Each steps of algorithm must be precisely defined. Every step of algorithm should be clear and not have any ambiguity.
- **Input:** Quantities which are given to it initially before the algorithm begins. All the algorithms should have some input. The logic of the algorithm should work on this input to give the desired result.
- **Output:** Quantities which have a specified relation to the input. At least one output should be produced from the algorithm based on the input given.
- **Effectiveness:** All the operations to be performed in the algorithm must be sufficient. Every step in the algorithm should be easy to understand and can be implemented using any programming language.

Performance Analysis and Measurement
(Continue…)

Example of an Algorithm:

- ***Write Algorithm to compute the average of n elements.***

1. **ALGORITHM: FIND AVERAGE OF N NUMBER**
2. Assume that array contains **n** integer values.
3. [INITIALIZE] **sum ←0.**
4. [PERFORM SUM OPERATION]
5. **Repeat step from i=0 to n-1**
6. **sum←sum+a[i]**
7. [CALCULATE AVERAGE]
8. **avg←sum/n**
9. Write **avg**
10. [FINISHED]
11. **Exit**

Performance Analysis and Measurement
(Continue…)

Complexity of Algorithms

1) Time Complexity:

- Running time of a program.
- Time Complexity is a function describing the amount of time an algorithm takes in terms of the amount of input to the algorithm.
- The **Time Complexity** of an algorithm is the amount of computer time it needs to execute the program and get the intended result.

Performance Analysis and Measurement
(Continue…)

2) Space Complexity:

- Amount of computer memory required during the program execution.
- **Space complexity** is a function describing the amount of memory (space) an algorithm takes in terms of the amount of input to the algorithm.
- Generally, the space needed by a program depends on the following two parts:
  - Fixed Part, that varies from problem to problem. In includes the space needed for storing instructions, constants, variables, and structures variables (like arrays and structures).
  - Variable Part, that varies from program to program. In includes the space needed for recursion stack, and for structured variables that are allocated space dynamically during the runtime of a program.

Performance Analysis and Measurement
(Continue…)

Performance Analysis of an Algorithm:

1) Worst Case Analysis(Big O notation)

- This denotes the behavior of an algorithm with respect to the worst possible of the input instance.
- In the worst case analysis, we calculate upper bound on running time of an algorithm.
- We must know the case that causes maximum number of operations to be executed.
- For Linear Search, the worst case happens when the element to be searched is not present in the array.
- When x is not present, the search () functions compares it with all the elements of array [] one by one.
- Therefore, the worst case time complexity of linear search would be.

Performance Analysis and Measurement
(Continue…)

Performance Analysis of an Algorithm:

2) Average Case Analysis(Theta Θ notation)

- The term "Average case running time of an algorithm is an estimate of the running time for an 'average' input.
- In average case analysis, we take all possible inputs and calculate computing time for all of the inputs.
- Sum all the calculated values and divide the sum by total number of inputs.
- We must know (or predict) distribution of cases.
- For the linear search problem, let us assume that all cases are uniformly distributed.
- So we sum all the cases and divide the sum by (n+1).

Performance Analysis and Measurement
(Continue…)

Performance Analysis of an Algorithm:

3) Best Case Analysis(Omega Ω notation)

- The term "Best case performance" is used to analyse an algorithm under optimal condition.
- In the best case analysis, we calculate lower bound on running time of an algorithm.
- We must know the case that causes minimum number of operations to be executed.
- In the linear search problem, the best case occurs when x is present at the first location.

Performance Analysis and Measurement
(Continue…)

Performance Analysis of an Algorithm:

Example:

- Assume there are 100 students in a class.
  - The worst case of the problem is none of the students clearing the exams.
  - The best of the problem is all 100 students clearing the exams.
  - The average case of the problem is around 50 students clearing the exams.

Thank You